

PRACTICAL 11

Q1. Multithreaded Counter - Race Condition

Aim

To develop a multithreaded counter using POSIX threads and demonstrate a race condition when multiple threads update a shared variable.

Theory

A race condition occurs when multiple threads access and modify shared data at the same time without synchronization.

pthread_create() creates a thread and pthread_join() waits for a thread to finish.

Program

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < 100000; i++) {
        counter++;
    }

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter = %d\n", counter);

    return 0;
}
```

Expected Output

```
amrutha@blue:~$ nano counter.c
amrutha@blue:~$ gcc counter.c -o counter
amrutha@blue:~$ ./counter
Final counter = 194447
```

Observation

Both threads modify the same counter variable simultaneously, causing a race condition.

Result

Thus, a race condition was successfully demonstrated using POSIX threads.

Q2. Counter Using Mutex

Aim

To modify the previous multithreaded counter program using mutex locks and compare it with the race-condition version.

Theory

A mutex allows only one thread at a time to access a critical section.

Important functions:

pthread_mutex_lock()

pthread_mutex_unlock()

Program

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter = %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

Output

```
amrutha@blue:~$ nano mutex.c
amrutha@blue:~$ gcc mutex.c -o mutex
amrutha@blue:~$ ./mutex
Final counter = 200000
```

Final counter = 200000

Comparison

Without Mutex	With Mutex
Race condition possible	Race condition prevented
Result may vary	Correct result
No synchronization	Uses synchronization

Observation

The mutex ensures that only one thread modifies the counter at a time.

Result

Thus, the race condition was prevented using mutex synchronization.